Lab: Operation Go Live

Modules Covered: M10 – M12 **Duration:** 90 – 120 minutes **Difficulty:** Beginner-Intermediate **Prerequisites:** A fresh Ubuntu VM (local or cloud) with sudo access

---

The Scenario

After onboarding the new crew, TechNova Corp's staging server is a fortress. Every account has proper permissions, SSH keys are in place, and sudo is locked down by role. There's just one problem — the server is completely empty. No web server, no database, no monitoring tools. Nothing is installed and nothing is running.

Your team lead Anas messages you:

> "The accounts are set up, SSH is solid, permissions are right. Great work. But right now that server is a very secure paperweight.
>
> Product needs a working demo by end of week. That means a web server serving pages, a database backend, and monitoring so we know when something breaks. I need you to install the full stack, get every service running, and make sure they survive a reboot.
>
> One more thing — don't just get it working and walk away. One server today, ten servers next month. Write a setup script so the whole process is repeatable. If we had to spin up a new box tomorrow, I want it done in minutes, not hours."

You open your terminal. An empty server, a deadline, and a mandate to automate.

Before You Start

This lab requires **no setup script**. Everything you do uses real Linux commands on a stock Ubuntu VM. You'll install packages, configure services, manage systemd units, and write a setup script.

Rules:

- Type every command yourself — no copy-paste
- When you see **"Try this (it will fail)"**, do it anyway — the error IS the lesson
- When you see a checkbox, pause and make sure you understand before moving on

---

Part 1: Stock the Arsenal (M10)

Your server is locked down and empty. Before anything can run, you need to install it. Linux uses a package manager called APT — think of it as an app store for servers.

Task 1.1 — What's already on this server?

Count how many packages are already installed on this "empty" server:

```
dpkg -l | wc -l
```

Now peek at a few of them:

```
apt list --installed | tail -10
```

Even a fresh Ubuntu install comes with hundreds of packages pre-installed — core utilities, networking tools, the kernel itself. You're not starting from zero; you're building on a foundation.

Task 1.2 — Refresh the menu

Before installing anything, update the package list:

```
sudo apt update
```

Watch the output. APT is downloading the latest catalog of available packages from Ubuntu's servers. It is NOT installing or upgrading anything — just refreshing what it knows is available.

> **Tip:** Think of it like refreshing a restaurant menu. `apt update` refreshes the menu. `apt install` places the order. Always refresh before you order.

Rule: **ALWAYS** run `apt update` before `apt install`. Skipping this step means you might install outdated versions or fail to find packages that exist.

Task 1.3 — Find the right package

Search for a web server:

```
apt search web server
```

That returns a lot of results. Narrow it down — inspect the one you want:

```
apt show nginx
```

Read the output. Write down:

- The version number
- The installed size
- How many dependencies it lists

This is your pre-install research. On production servers, you check versions and sizes before installing anything.

Task 1.4 — Install your first package

Install nginx:

```
sudo apt install nginx
```

Watch the output carefully. APT:

1. Resolved dependencies — figured out everything nginx needs
2. Showed you the download size and disk usage
3. Asked for confirmation
4. Downloaded, unpacked, and configured every package

One command. All dependencies handled automatically.

Task 1.5 — Try without sudo (it will fail)

Try installing a package without sudo:

```
apt install htop
```

You should see an error like `Could not open lock file` or `Permission denied`.

Installing software changes system files that affect every user on the machine. That's why it requires root privileges. This is the same principle you learned with user management — dangerous operations need `sudo`.

Task 1.6 — Install the full stack

Install multiple packages in one command:

```
sudo apt install -y postgresql htop tree curl
```

The `-y` flag auto-answers "yes" to the confirmation prompt. You'll use this in scripts where there's no human to type `y`.

Installing multiple packages in one command is faster than installing them one at a time — APT resolves all dependencies together and avoids redundant downloads.

Task 1.7 — Inspect what you installed

List the files that nginx installed on your system:

```
dpkg -L nginx | head -20
```

Now go the other direction — given a file, find which package owns it:

```
dpkg -S /usr/sbin/nginx
```

Every file on a managed system traces back to a package. If you find a mysterious binary, `dpkg -S` tells you where it came from. If you need to know what a package put on your disk, `dpkg -L` lists every file.

Task 1.8 — Remove and clean up

Install something small to practice removal:

```
sudo apt install -y cowsay
```

Test it:

```
/usr/games/cowsay "TechNova is live"
```

Now remove it:

```
sudo apt remove cowsay
```

That removed the program, but its configuration files are still on disk. Purge removes everything:

```
sudo apt purge cowsay
```

Finally, clean up any dependencies that were installed for cowsay but are no longer needed by anything:

```
sudo apt autoremove
```

Command	What it removes
`apt remove`	The package binary, keeps config files
`apt purge`	The package binary AND config files
`apt autoremove`	Orphaned dependencies no package needs

Task 1.9 — Where does software come from?

Check where APT downloads packages from:

```
cat /etc/apt/sources.list
```

These are your repositories — remote servers that host packages. Ubuntu organizes them into categories:

Repository	What it contains
main	Officially supported, open-source packages
universe	Community-maintained, open-source packages
restricted	Proprietary drivers (e.g., NVIDIA)
multiverse	Non-free software (codecs, fonts)

> **Production tip:** Stick to **main** and **universe** for production servers. Third-party repos can introduce untested or conflicting packages. Every repo you add is a trust decision.

Checkpoint: Part 1

- [ ] You updated package lists with `apt update`
- [ ] You searched for and installed packages with `apt search` and `apt install`
- [ ] You know why `sudo` is required for installation
- [ ] You inspected package contents with `dpkg -L` and `dpkg -S`
- [ ] You understand `remove` vs `purge` vs `autoremove`
- [ ] You know where packages come from (repositories)

Part 2: Bring It to Life (M11)

Software is installed. But installed does not mean running. A web server sitting on disk serves nobody. Time to bring it to life.

Task 2.1 — Is it running?

Check the status of the nginx service you just installed:

```
systemctl status nginx
```

Read the output carefully. Every line tells you something:

Field	What it means
Loaded	The unit file exists and systemd knows about it
enabled/disabled	Whether it starts automatically on boot
Active	Whether the process is running right now
Main PID	The process ID of the running service

> **Tip:** On Ubuntu, `apt install nginx` may auto-start it. That's the package's install script doing it, not something you configured. Other distros or packages may not auto-start — never assume.

Task 2.2 — Stop the service

Stop nginx:

```
sudo systemctl stop nginx
```

Verify it's actually stopped:

```
systemctl is-active nginx
```

You should see `inactive`. Now try to reach it:

```
curl localhost
```

Connection refused. The software is still installed — every file is still on disk — but nothing is listening. "Installed" and "running" are completely different states.

Task 2.3 — Start it back up

Bring nginx back to life:

```
sudo systemctl start nginx
```

Verify:

```
systemctl is-active nginx
```

You should see `active`. Now test it:

```
curl localhost
```

You should see HTML output — the default "Welcome to nginx!" page. Write down what you see. That default page means the web server is installed, running, and serving content to anyone who connects.

Task 2.4 — Reload vs restart

These two commands sound similar but behave very differently:

```
sudo systemctl reload nginx
```

```
sudo systemctl restart nginx
```

Action	What happens	Downtime	When to use
reload	Re-reads config files, keeps process running	None	After config changes
restart	Stops the process, starts a new one	Brief	After binary updates or when reload fails

> **Production tip:** Always prefer `reload` when possible. A restart drops every active connection for a moment. On a server handling thousands of requests, that moment matters.

Task 2.5 — Survive a reboot

Check whether nginx is set to start automatically on boot:

```
systemctl is-enabled nginx
```

Disable it — nginx will still run right now, but won't start after a reboot:

```
sudo systemctl disable nginx
```

Enable it again:

```
sudo systemctl enable nginx
```

You can enable AND start in one command:

```
sudo systemctl enable --now nginx
```

A service can be **running but not enabled** (it works now but won't survive a reboot) or **enabled but not running** (it will start on boot but isn't active right now). These are independent states. Getting a service "production ready" means both: running AND enabled.

Task 2.6 — Read the service logs

View the last 20 log entries for nginx:

```
sudo journalctl -u nginx -n 20
```

Filter by time — only the last 5 minutes:

```
sudo journalctl -u nginx --since "5 minutes ago"
```

Follow the log in real-time (press Ctrl+C to stop):

```
sudo journalctl -u nginx -f
```

> **Tip:** When a service won't start, `journalctl -u servicename` is always the first place to look. The error message is almost always in the log.

Task 2.7 — The service is down (it will fail)

Simulate an outage. Stop nginx:

```
sudo systemctl stop nginx
```

Try to reach it:

```
curl localhost
```

Connection refused. Check the status:

```
systemctl status nginx
```

You should see `inactive (dead)`. Now fix it:

```
sudo systemctl start nginx
```

Verify the fix:

```
curl localhost
```

It works again. This is the core production loop: **check status, read logs, fix, verify**. You will repeat this sequence hundreds of times in your career.

Task 2.8 — What else is running?

List every active service on this machine:

```
systemctl list-units --type=service
```

That's a lot of services. Now check if anything is broken:

```
systemctl list-units --type=service --state=failed
```

Ideally you see zero failed services. You'll likely recognize some names in the full list — `sshd` (remote access), `cron` (scheduled tasks), `postgresql` (database you installed in Part 1). Every one of them is managed by systemd the same way you just managed nginx.

Task 2.9 — Under the hood: unit files

View the unit file that tells systemd how to manage nginx:

```
systemctl cat nginx
```

A unit file has three sections:

Section	Purpose
[Unit]	What this service is and what it depends on
[Service]	How to start, stop, and restart the process
[Install]	When to start it (which boot target)

Key directives you'll see:

- `ExecStart` — the command that launches the process
- `Restart` — what to do if the process crashes
- `After` — which services must start before this one
- `WantedBy` — which target pulls this service in at boot

> **Tip:** Custom services go in `/etc/systemd/system/`. You'll create your own unit files in later modules. For now, just know where to look when you need to understand how a service is configured.

Checkpoint: Part 2

- [ ] You can start, stop, and restart services with `systemctl`
- [ ] You know the difference between `reload` and `restart`
- [ ] You can enable and disable services for boot with `systemctl enable/disable`
- [ ] You can read service logs with `journalctl -u`
- [ ] You understand unit files and their three sections
- [ ] You experienced "installed but not running" firsthand

Part 3: Write It Down (M12)

You just spent 30 minutes typing commands to install packages and configure services. What happens when you need to set up a second server? A third? Ten? Time to write it down — in a script.

Task 3.1 — Your first script

Open a new file in nano:

```
nano ~/system-info.sh
```

Type this content exactly:

```
#!/bin/bash
set -e
echo "═══ System Information ═══"
echo "Hostname: $(hostname)"
echo "Date: $(date)"
echo "User: $USER"
```

Save and exit: Ctrl+O, Enter, Ctrl+X.

Two lines at the top matter more than the rest of the script combined:

- `#!/bin/bash` is the **shebang**. It tells Linux which interpreter to use when running this file. Without it, the system doesn't know if this is bash, Python, or something else.
- `set -e` makes the script stop immediately if any command fails. Without it, bash happily continues after errors — silently producing wrong results while you think everything is fine.

Task 3.2 — Make it run (it will fail first)

Try running your script:

```
./system-info.sh
```

Permission denied. You wrote the file, but Linux doesn't consider it executable yet. Fix it:

```
chmod +x ~/system-info.sh
```

You learned `chmod` in M06 — callback to Operation Lockdown. The `+x` flag adds execute permission, telling Linux this file is allowed to run as a program.

Now run it again:

```
./system-info.sh
```

You should see output like:

```
═══ System Information ═══
Hostname: technova-staging
Date: Tue Mar 4 09:15:22 UTC 2026
User: anas
```

Why do you need `./` in front? Linux only looks for commands in directories listed in `$PATH`. Your current directory is not in `$PATH` — intentionally, for security. If it were, an attacker could drop a malicious `ls` in your home folder and you'd run it by accident. The `./` prefix means "look right here in this directory."

Task 3.3 — Variables and command substitution

Your script reports three facts. A real system report needs more. Edit the script:

```
nano ~/system-info.sh
```

Add these lines after the existing echo statements (before the closing of the file):

```
UPTIME=$(uptime -p)
MEMORY=$(free -h | awk 'NR==2 {print $3 "/" $2}')
DISK=$(df -h / | awk 'NR==2 {print $5}')
PACKAGES=$(dpkg -l | wc -l)
SERVICES=$(systemctl list-units --type=service --state=running --no-pager |

echo "Uptime:        $UPTIME"
echo "Memory:        $MEMORY"
echo "Disk usage:    $DISK"
echo "Packages:      $PACKAGES"
echo "Running svcs:  $SERVICES"
echo "═══════════════════════"
```

Save, exit, and run it:

```
./system-info.sh
```

Now your script reports real system data — uptime, memory, disk, package count, and running services. All from one command.

The key here is `$()` — **command substitution**. The command inside the parentheses runs first, its output replaces the `$( ... )` expression, and that value gets stored in the variable. Think of it as "run this, capture the result."

Task 3.4 — Quoting matters

Try these three commands in the terminal (not in a script — this is an interactive demonstration):

```
NAME="TechNova Corp"
echo "Welcome to $NAME"
echo 'Welcome to $NAME'
echo Welcome to $NAME
```

Three different results:

- **Double quotes:** Variable expands — you see `Welcome to TechNova Corp`
- **Single quotes:** Everything is literal — you see `Welcome to $NAME`
- **No quotes:** Variable expands and it looks fine here, but no quotes means word splitting. With filenames or values containing spaces, unquoted variables break silently.

Quote Type	Variables Expand?	Spaces Safe?	Use When
`"double"`	Yes	Yes	Most of the time
`'single'`	No	Yes	Literal strings
None	Yes	No	Almost never

Rule: when in doubt, use double quotes. Always quote `"$VARIABLES"`.

Task 3.5 — Special variables

Edit your script one more time:

```
nano ~/system-info.sh
```

Add these lines at the end, before the `═══════════════════` line:

```
echo "Script name:   $0"
echo "Arguments:     $#"
echo "Working dir:   $PWD"
```

Save and run it with an argument:

```
./system-info.sh test
```

Then run it without:

```
./system-info.sh
```

Notice that `$#` changes — it's `1` with the argument, `0` without. These are **special variables** that the shell sets for you automatically:

- `$0` — the script's own name
- `$#` — how many arguments were passed
- `$@` — all arguments as a list
- `$PWD` — current working directory
- `$HOME` — home directory
- `$USER` — current username

You've already been using `$USER` since Task 3.1. These variables are always available in every script and every interactive session — the shell populates them for you.

Task 3.6 — The mission: automate everything

You just installed packages by hand, started services by hand, and verified by hand. That worked for one server. But your team lead said it: one server today, ten servers next month. Time to automate.

Create a new script:

```
nano ~/setup-server.sh
```

Start with this:

```
#!/bin/bash
set -e

# Configuration
PACKAGES="nginx curl htop tree"
SERVICES="nginx"
```

Save and exit (Ctrl+O, Enter, Ctrl+X). You'll keep adding to this file over the next four tasks.

Notice the pattern: variables at the top, logic below. Change one line — the `PACKAGES` variable — and you change what gets installed. You never have to dig through the script to modify behavior. This is the same pattern used in every infrastructure-as-code tool: Ansible has variable files, Terraform has `.tfvars`, CloudFormation has parameters. Configuration at the top, execution below.

Task 3.7 — Add the package installation block

Open the script again:

```
nano ~/setup-server.sh
```

Add these lines below the configuration section:

```
# Update package lists
echo "Updating package lists..."
sudo apt update

# Install packages
echo "Installing packages: $PACKAGES"
sudo apt install -y $PACKAGES
```

Save and exit.

Two details matter here. First, the `echo` before each step. When a script runs 50 commands, you need to know where it is. These progress messages tell you what's happening right now — and more importantly, which step it was on when something went wrong. Second, the `-y` flag. You used it in Task 1.6 for convenience. In a script, it's mandatory. Without `-y`, the script pauses at the confirmation prompt and waits forever for a human who isn't watching.

Task 3.8 — Add the service management block

Open the script again:

```
nano ~/setup-server.sh
```

Add below the package installation block:

```
# Enable and start services
for SERVICE in $SERVICES; do
    echo "Enabling and starting $SERVICE..."
    sudo systemctl enable --now "$SERVICE"
done
```

Save and exit.

This uses a `for` loop. You'll learn loops properly in M13, but the pattern is readable right now: "for each service in the list, enable and start it." The `enable -now` combination does both — sets it to start on boot AND starts it immediately. You used this exact command in Task 2.5.

One detail: `"$SERVICE"` is in double quotes. This protects against service names that might contain spaces. They almost never do, but quoting variables is a habit that prevents bugs in scripts where values are less predictable.

Task 3.9 — Add the verification block

Open the script one more time:

```
nano ~/setup-server.sh
```

Add at the bottom:

```
# Verify
echo ""
echo "═══ Verification ═══"
echo "Nginx status: $(systemctl is-active nginx)"
echo "Nginx enabled: $(systemctl is-enabled nginx)"
echo "Web server test:"
curl -s -o /dev/null -w "  HTTP status: %{http_code}\n" http://localhost
echo ""
echo "Setup complete!"
```

Save and exit.

The `curl` line looks dense, but each flag does one thing:

Flag	What it does
`-s`	Silent mode — no progress bar
`-o /dev/null`	Discard the HTML body (you don't need to see it)
`-w "..."`	Print a custom format string instead
`%{http_code}`	The HTTP status code from the response

If you see `HTTP status: 200`, the web server is responding. This is how you verify a web server in a script — you don't read HTML, you check the status code.

Task 3.10 — Run the capstone

Make the script executable:

```
chmod +x ~/setup-server.sh
```

Before running it, undo your manual work so you can test the script on a "clean" state. Stop nginx:

```
sudo systemctl stop nginx
```

Verify it's down:

```
curl localhost
```

You should see `Connection refused`. Good — the service is down, simulating a server that needs setup.

> **Tip:** Since you already installed these packages in Part 1, `apt install` will confirm they're already at the latest version rather than downloading them again. On a truly fresh server, you'd see the full download and installation output. The script handles both cases identically.

Now run the script:

```
./setup-server.sh
```

Watch it work. Every step prints what it's doing. Packages update, software installs, services enable and start, verification runs. Everything you did manually across Parts 1 and 2 — 30+ minutes of typing individual commands — happens in seconds.

Test the result:

```
curl localhost
```

Nginx is serving again. Not because you typed 15 commands. Because you ran one script.

> **Tip:** Save this script. On a fresh Ubuntu VM, you can run it and have a configured web server in under a minute. Add more packages to the `PACKAGES` variable, more services to the `SERVICES` variable, and the script scales with you. This is the same idea behind Ansible, Terraform, and CloudFormation — write what you want, run the script, get a configured server. You just wrote your first piece of infrastructure as code.

Task 3.11 — Source vs execute

Your setup script runs with `./setup-server.sh`. But what if a script sets variables instead of running commands? Create a small config file:

```
nano ~/config.sh
```

Type this content:

```
#!/bin/bash
APP_ENV="staging"
APP_PORT=3000
```

Save and exit (`Ctrl+O` , `Enter` , `Ctrl+X`). Make it executable:

```
chmod +x ~/config.sh
```

Now execute it the normal way:

```
./config.sh
echo $APP_ENV
```

Empty. Nothing prints. The script ran in a **child shell** — a separate process that started, created those variables, and then exited. When it exited, everything it created went with it.

Now try sourcing it instead:

```
source ~/config.sh
echo $APP_ENV
```

"staging". The variables are in your current shell.

The difference is fundamental:

- `./script.sh` — runs in a **new (child) shell**. When it exits, everything is gone.
- `source script.sh` — runs in your **current shell**. Variables, functions, and settings persist.

Real-world pattern: **source** config files (you want their variables). **Execute** action scripts (you don't want them polluting your shell).

Task 3.12 — Export variables

Set a variable in your terminal:

```
NAME="TechNova"
bash -c 'echo $NAME'
```

Empty. You launched a child process (`bash -c`), and it can't see `NAME`. The variable is **shell-local** — it only exists in the shell where you defined it.

Now export it:

```
export NAME="TechNova"
bash -c 'echo $NAME'
```

"TechNova". The child process can see it now.

Verify it's in the environment:

```
env | grep NAME
```

You'll see `NAME=TechNova` in the output.

Without `export`, a variable is shell-local — only your current shell can see it. With `export`, it becomes an **environment variable** visible to anything you launch: child shells, scripts, programs. This is how applications like nginx and PostgreSQL read their configuration — through exported environment variables.

Task 3.13 — Write the deployment report

Every deployment should leave a paper trail. Use redirection to compile a report from real system data:

```
echo "═══ DEPLOYMENT REPORT ═══" > ~/deployment-report.txt
echo "Date: $(date)" >> ~/deployment-report.txt
echo "Operator: $USER" >> ~/deployment-report.txt
echo "" >> ~/deployment-report.txt
echo "--- Installed Packages ---" >> ~/deployment-report.txt
dpkg -l nginx postgresql curl htop tree 2>/dev/null | grep "^ii" >> ~/deplo
echo "" >> ~/deployment-report.txt
echo "--- Service Status ---" >> ~/deployment-report.txt
systemctl is-active nginx >> ~/deployment-report.txt
echo "" >> ~/deployment-report.txt
echo "--- Web Server Test ---" >> ~/deployment-report.txt
curl -s -o /dev/null -w "HTTP %{http_code}" http://localhost >> ~/deploymen
echo "" >> ~/deployment-report.txt
echo "═══ END REPORT ═══" >> ~/deployment-report.txt
```

Review what you built:

```
cat ~/deployment-report.txt
```

You should see a clean report with the date, your username, installed package versions, service status, and an HTTP response code. All generated from live system data, all assembled with redirection.

Reminder: `>` **overwrites** the file (that first line creates it fresh). `>>` **appends** to the file (every line after adds to it). `2>/dev/null` silences error output so missing packages don't clutter the report.

Checkpoint: Part 3

- [ ] You wrote and executed bash scripts with `chmod +x` and `./script.sh`
- [ ] You used variables to store data (`NAME="value"`)
- [ ] You captured command output with `$(command)`
- [ ] You understand quoting: double expands variables, single is literal
- [ ] You know the difference between source and execute
- [ ] You automated manual work into a repeatable setup script
- [ ] You used `set -e` to make scripts fail fast

Final Verification

Run this block to review everything you built during this lab:

```
echo "═══ Operation Go Live --- Final Check ═══"
echo ""
echo "--- Packages ---"
dpkg -l nginx postgresql curl htop tree | grep "^ii" | awk '{print $2, $3}'
echo ""
echo "--- Services ---"
systemctl is-active nginx
systemctl is-enabled nginx
echo ""
echo "--- Web Server ---"
curl -s localhost | head -5
echo ""
echo "--- Scripts ---"
ls -l ~/system-info.sh ~/setup-server.sh
echo ""
echo "--- Deployment Report ---"
cat ~/deployment-report.txt
```

If everything prints cleanly — packages with version numbers, services active and enabled, HTML from localhost, your scripts with execute permissions, and a complete deployment report — the operation is a success.

What You Accomplished

You started with an empty server and a deadline. No web server, no database, no monitoring, no automation. Just a secure box with nothing on it.

You built all of it:

1. **Installed the full application stack** with APT — web server, database, monitoring tools, and utilities. You learned to search, install, inspect, and remove packages.
2. **Configured services** to run and survive reboots with systemd. You started, stopped, restarted, reloaded, enabled, and disabled services. You read their logs and understood their unit files.
3. **Wrote a system info script** to learn bash fundamentals — variables, command substitution, quoting, special variables, and making scripts executable.
4. **Built a setup script** that automates the entire installation and configuration process in seconds. What took 30 minutes of manual commands now runs with a single `./setup-server.sh`.
5. **Documented everything** in a deployment report assembled from live system data using redirection.

> The same pattern you just followed — install software, manage services, automate with scripts — is the foundation of every cloud deployment. APT is the same concept as npm, pip, and Docker Hub. systemd is what Docker restart policies and Kubernetes liveness probes evolved from. Your setup script is the ancestor of Ansible playbooks, Terraform configs, and CloudFormation templates. Same concepts, bigger scale.
>
> Next: your scripts can run commands. But what if nginx isn't installed? What if a file doesn't exist? What if a service won't start? Time to add logic — decisions and loops that make scripts smart.

Bonus Challenges

Finished early? Try these without looking at the instructions above.

Challenge 1: Version Check Script

Write a script called `versions.sh` that displays the version of nginx and postgresql.

Hint: `nginx -v 2>&1`, `psql -version`

▶ Solution

Challenge 2: Service Watchdog

Write a script that checks if nginx is running. If it's not, start it and log the event.

Hint: `systemctl is-active nginx` *returns "active" or "inactive"*

▶ Solution

Challenge 3: Package Audit

Write a one-liner that finds all installed packages with "lib" in the name and counts them.

Hint: pipe `dpkg -l` *through* `grep`

▶ Solution

Challenge 4: Argument-Driven Script

Modify setup-server.sh to accept a package name as `$1` and install just that one package.

Hint: check if `$1` *is provided, then use it*

▶ **Solution**

Command Reference

Commands

Command	What It Does	Module
`apt update`	Refresh available package lists	M10
`apt install`	Install packages with dependencies	M10
`apt remove`	Remove package (keep config)	M10
`apt purge`	Remove package and config files	M10
`apt autoremove`	Remove unused dependencies	M10
`apt search`	Find packages by keyword	M10
`apt show`	Show package details	M10
`dpkg -l`	List all installed packages	M10
`dpkg -L`	List files installed by a package	M10
`dpkg -S`	Find which package owns a file	M10
`systemctl status`	Show service status and recent logs	M11
`systemctl start`	Start a service	M11
`systemctl stop`	Stop a service	M11
`systemctl restart`	Stop then start a service	M11
`systemctl reload`	Reload config without downtime	M11
`systemctl enable`	Start service on boot	M11

Command	What It Does	Module
`systemctl disable`	Don't start service on boot	M11
`systemctl is-active`	Check if service is running	M11
`systemctl is-enabled`	Check if service starts on boot	M11
`systemctl list-units`	List all active units	M11
`systemctl cat`	View unit file contents	M11
`journalctl -u`	Read logs for a service	M11
`chmod +x`	Add execute permission	M12
`set -e`	Exit script on first error	M12
`$()`	Command substitution	M12
`export`	Make variable available to child processes	M12
`source`	Run script in current shell	M12

Symbols

Symbol	What It Does	Module
`#!/bin/bash`	Shebang — specifies interpreter	M12
`$VAR`	Variable expansion	M12
`"double quotes"`	Expand variables, preserve spaces	M12
`'single quotes'`	Everything literal	M12
`>`	Redirect output (overwrite)	M12
`>>`	Redirect output (append)	M12

Lab designed for the Linux Fundamentals for Cloud Computing course.